

N-way Set-Associative Cache

Description

The purpose of this exercise is to implement a N-Way Set-Associative Cache that is accessible to users through a library.

This cache should support all types in Java for its keys and values, and an instantiation of stated cache must have a declared key and value type, which can be of different types. All keys and values should be the specified type when reading/writing from the cache, otherwise exceptions will be thrown. Support for all keys and values should also be extended to cache replacement algorithms. The library should also include replacement algorithm flexibilities for users.

Approach

The purpose of a cache is to reduce the time it takes for users to retrieve viable information without having to access database and outside servers, and within the constraints of a cache being viable until the end of code execution, this translates to using methods within this library to obtain a faster access time for users during run time. With this consideration in mind, this design opted to utilize the $O(1)$ retrieval time of a HashMap data structure in Java.

The process of retrieving data from this cache follows these steps:

1. Generate a hash integer between 0 and N-1 (inclusive) for a given key
2. Within the outer hashMap, access the inner hashMap associated with the generated hashCode => $O(1)$
3. Retrieve the value stored using the given key from the selected inner hashMap => $O(1)$
4. Return the value to the user

Technologies

JDK 8+, JUnit, Apache Ant

Structure

Compile and use the library by using the following steps:

1. Compile associated library files in the project directory `/cachelibrary`

```
ant jar
```

2. Move the created jar file `NSetCache.jar` to the same directory as your main usage class.

3. Compile main execution file with `.jar` file

```
javac -cp NSetCache.jar Example.java
```

4. Extract the class files from the jar files

```
jar xf NSetCache.jar
```

5. Execute main file

```
java Example
```

To compile this project:

```
ant compile
```

To run the unit tests for this project:

```
ant test
```

To clean the project of executables:

```
ant clean
```

Data Layer

- The Cache will use a nested HashMap data structure that looks like this:

```
(0) => array(key => CacheItem, key => CacheItem),  
...  
(n-1) => array(key => CacheItem, key => CacheItem)
```

- The hashing algorithm for this Cache utilizes the MD5 Hashing Algorithm with the object's hashcode inputted as a parameter along with the modulo operation with (N-1). The reason for this design choice is to ensure that equal (i.e. identical member variables) objects, despite different pointer references in the Java heap, will always result in the same hash.
- Note that different objects can result in the same hash as well, this is called a "collision".

This NSetCache will utilize Least-Recently Used(LRU) and Most-Recently Used(MRU) replacement algorithms to resolve collisions when there are no more entries in the corresponding hash's block.

- The Java library defines `.equals()` and `.hashCode()` functions for primitive types, however, it is the user's responsibility to override `.equals()` and `.hashCode()` functions for their respective custom object classes to ensure that utilizing objects as keys in this Cache will result in correct retrievals. For more information on the interaction between `.hashCode()` and `.equals()`, you can refer to [this](#). HashMap utilizes the object's `.equals()` function to compare whether or not the given key is equal or not. For more information on the internal structure of a hashmap, refer to [this](#).

CacheLibrary Interface

```
public interface CacheLibrary<K,V>
{
    /**
     * @brief Retrieves a value from the cache by a given key
     * @return V Value corresponding to given key
     */
    public CacheItem get(K key);

    /**
     * @brief Inserts a key-value pair into the cache
     * @param K key
     * @param V value
     * @return boolean True on success
     */
    public boolean set(K key, V value);

    /**
     * @brief Deletes a key-value entry from the cache by a given key
     * @param K key
     * @return boolean True on success
     */
    public boolean delete(K key);

    /**
     * @brief Retrieves key-value pairs from the cache by the given keys
     * @return HashMap<K,V> collection
     */
    public HashMap<K,CacheItem> getCollection(K[] keys);

    /**
     * @brief Inserts key-value pairs into the cache
     * @param HashMap<K,V> entries
     */
    public boolean setCollection(HashMap<K, V> entries);

    /**
```

```

    * @brief Deletes key-value entries from the cache that corresponds to given keys
    * @param K key
    * @return boolean True on success
    */
    public boolean deleteCollection(K keys[]);

    /**
     * @brief Clears all entries from the cache
     */
    public void clear();

    /**
     * @brief Evicts key-value pairs when cache is full and an insertion is required
     */
    public void eviction(int index);
}

```

CacheItem Class

Class used by NSetCache to store a cache entry as an object.

Instant class is used to represent the timestamp of cache creation, because it is thread-safe and has a higher level of precision compared to milliseconds.

```

public class CacheItem<K,V>
{
    private Instant timestamp;
    private K key;
    private V value;

    public CacheItem(K key, V value)
    {
        this.key = key;
        this.value = value;
        this.timestamp = Instant.now();
    }

    public CacheItem(K key, V value, long timestamp)
    {
        this.key = key;
        this.value = value;
        this.timestamp = timestamp;
    }

    public V getValue()
    {
        return this.value;
    }

    public K getKey()
    {

```

```

        return this.key;
    }
    public Instant getTimestamp()
    {
        return this.timestamp;
    }
    public void updateTimestamp()
    {
        this.timestamp = Instant.now();
    }
}

```

NSetCache Base Class implementing the interface

Users are able to override the functions in this class if desired to modify the hashing function and the replacement algorithms. The second constructor consisting of a fourth parameter, `function`, allows users to define the function the cache should invoke when inserting while the cache is full. The expected parameter in this function are the defined string constants in this class. Aside from the provided LRU/MRU replacement algorithm, the `userDefinedAlgorithm` is defaulted to LRU to enable general flexibility for users to override and define their own replacement criterias and can be extendable for further specific usage when overridden.

```

public class NSetCache
{
    // constants of replacement algorithm options
    public static final String LEAST_RECENT = "LRU";
    public static final String MOST_RECENT = "MRU";
    public static final String USER_DEFINED = "USER DEFINED";

    // private variables
    private final int N_sets;
    private final int M_blocks;
    private final String name;
    private final String method;
    private final HashMap<Integer, HashMap<K, CacheItem>> cache = new HashMap<Integer,

    /** mplements all interface methods */

    /**
     * @brief Basic Constructor
     * @param String name The name of the cache
     * @param Integer n The number of sets and entries in each set
     */
    public NSetCache(String name, Integer n, Integer m) throws IllegalArgumentException { ... }

    /**
     * @brief Constructor

```

```

    * @param String name The name of the cache
    * @param Integer n The number of sets
    * @param Integer m The number of entries per set
    * @param String function The name of the algorithm to invoke when the cache is full
    */
    public NSetCache(String name, Integer n, Integer m, String function) throws IllegalArgumentException { ... }

    /**
     * @brief Simple LRU Replacement Algorithm
     * @param int Index The index of the block to evict an entry from
     */
    public void LRU(int index)
    { ... }

    /**
     * @brief Simple MRU Replacement Algorithm
     * @param int Index The index of the block to evict an entry from
     */
    public void MRU(int index)
    { ... }

    /**
     * @brief Client usage function intended to be overridden with a custom replacement algorithm
     * @param int Index The index of the block to evict an entry from
     */
    public void userDefinedReplacementAlgorithm(int index)
    { ... }

    /**
     * @brief Retrieves a hashCode based on the key provided
     * @param K key
     * @return Integer The index of the block in which this key is mapped to (between 0 and n-1)
     */
    public int getHash(K key)
    { ... }
}

```

Scalability

This is a general usage cache with lots of flexibility allowed for users and will only support N blocks with M entries for cache data storage. It will be the user's responsibility to allocate an appropriate amount of N blocks upon cache instantiation for their specific use case.

Risk Analysis

Because the following library does not utilize any database or data structure connections of

any sort and will be distributed in similar form of a `.jar` file, the library source code is protected and no server/database overload will occur on our end. The data contained in this cache will only be viable until the end of code execution.

Deliverables

- ☐ CacheLibrary Interface
- ☐ NSetCache Class implements CacheLibrary
- ☐ CacheItem Class
- ☐ Unit Tests

Unit Test Checklist

- ☐ NSetCache Base Class functions
- ☐ Cache Library compilation/import usage

Task Breakdown

- ☐ CacheItem Class
- ☐ CacheLibrary Interface
- ☐ NSetCache: CacheLibrary interface implementations
- ☐ NSetCache: Hashing implementations
- ☐ NSetCache: LRU/MRU implementations
- ☐ Unit Tests